UCS503P - Software Engineering Lab

InvestCircle

A Peer-to-Peer Stock Research and Community Platform

Submitted by:

1024160099 Ishan Jha
1024160104 Lakshay Sachdeva
1024160106 Aarav K. Arora
1024160120 Anirudh Phophalia

Group: BE Third Year, CSE

Submitted to:

Anushka Ma'am

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**

1.1 Background and Context 3

1.1.1 Problem Statement 4

1.2 Project Objectives 4

2 Methodology and Design **6**

2.1 System Architecture 6

2.1.1 High-Level Design 6

2.1.2 Technical Stack 15

2.2 Implementation Details 16

2.2.1 Module 1: Core Functionality 16

2.2.2 Module 2: Secondary Features 19

3 Results and Evaluation **22**

3.1 Testing Methodology 22

3.2 Performance Metrics 22

3.3 Analysis 23

4 Conclusion **25**

4.1 Summary of Work 25

4.2 Key Findings 25

4.3 Future Work and Recommendations 26

4.4 Final Remarks 26

Bibliography **28**

Chapter 1

Introduction

1.1 Background and Context

Retail participation in Indian equity markets has grown rapidly, and with it the volume of informal investment discussion on social media, forums, and dedicated "tip" channels. However, these channels share a structural weakness: opinions and predictions are published freely, but almost never tracked. A contributor who is right ten times and wrong ten times looks, on the surface, identical to one who has never been wrong — because neither platform keeps a record. Follower counts and engagement metrics end up substituting for the one signal that actually matters: whether a person's calls have historically been accurate.

The higher-order goal of **InvestCircle** is therefore to create a more accountable and transparent environment for peer-to-peer financial research. Retail investors currently depend on social media, discussion forums, financial news websites and informal stock-tip communities, where opinions are often fragmented and historical predictions are rarely tracked. InvestCircle aims to translate the idea of accountable financial discussion into a measurable software system: instead of evaluating a contributor only through followers, likes or engagement, the platform maintains a persistent history of structured stock calls and their eventual outcomes.

The project is focused on financial research and discussion rather than trading. It will not execute trades or connect users to brokerage accounts.

Before finalising the project's scope, the existing landscape of tools retail investors already use was reviewed. General-purpose social platforms (Twitter/X threads, Reddit communities, Telegram "tip" groups) offer high engagement and a low barrier to publishing, but no platform-level mechanism ties a prediction to its author's historical record. Financial news and data platforms such as Moneycontrol, Economic Times Markets, and TradingView provide reliable market data, and TradingView in particular supports user-published trade ideas, but resolution of a published idea against its stated target is not a first-class, platform-tracked feature. Watchlist-centric broker and screener apps let a user track tickers they care about, but are single-player by design — there is no social graph and no publishing of research, and therefore nothing to hold accountable. No reviewed platform combines identity-based publishing, a structured machine-checkable prediction format, automatic resolution against real market data, and a persistent public accuracy record. That combination is the gap InvestCircle is designed to fill.

1.1.1 Problem Statement

Retail investors researching Indian equities commonly combine financial news websites, social-media discussions and informal stock-tip communities when researching investments. Although these platforms provide large amounts of information, they have several concrete limitations:

- **Lack of accountability** — users can repeatedly publish stock predictions without maintaining a visible record of previous outcomes.
- **Fragmented research** — financial news, stock discussions and individual research are distributed across multiple platforms.
- **Unstructured opinions** — many investment opinions are written as free-form text, making explicit predictions difficult to identify and evaluate.
- **Weak credibility signals** — follower counts and engagement do not necessarily represent historical prediction accuracy.
- **Limited personalisation** — users interested in specific stocks or sectors must often manually search for relevant news.
- **Poor historical visibility** — unsuccessful predictions may disappear from attention while successful predictions remain visible.
- **Doomscrolling without substance** — short-form scrolling formats (reels) are highly engaging but rarely used for anything meaningful, while genuinely useful discussion platforms are rarely built to be scrollable.

InvestCircle addresses these problems through identity-based participation, structured call tracking, public performance history, watchlists and personalised, scrollable financial content in one platform.

1.2 Project Objectives

1. **Objective 1:** Deliver a platform where users publish stock research with an attached, structured call (ticker, direction, target price, horizon date), such that all Must-priority functional requirements are demonstrated end-to-end at final evaluation.
2. **Objective 2:** Implement automatic call resolution against end-of-day market price data, resolving at least 95% of due calls without manual intervention, and surface the resulting accuracy score on each contributor's profile.
3. **Objective 3:** Demonstrate solid full-stack software engineering practice — relational schema design, REST API design, authentication, caching, and background job processing — with at least 70% automated test coverage on the backend business-logic layer and feed/profile pages loading within 1.5 seconds under seeded load.
4. **Objective 4:** Keep AI features strictly secondary, as an isolated optional microservice, so that all core flows remain fully functional when the AI service is unavailable.

These objectives are stated in SMART terms — each is specific, has a measurable threshold, is achievable within the project's six-month window, is directly relevant to the accountability problem defined above, and is bound to the milestone schedule described in Section 2.2.

The following are **in scope** for the first release: user accounts (signup/login, profile, bio, followed sectors); posts consisting of research notes with an optional structured call; public or private watchlists of tickers; follow/unfollow and a personalised feed of followed users' posts; comments and threaded replies; accuracy tracking via automatic call resolution; in-app notifications for follows, comments and resolved calls; search across users, tickers and posts; and an AI bolt-on that auto-tags each post with sector and sentiment.

The following are explicitly **out of scope** for the first release: real brokerage integration or real-money trading; native mobile applications (the platform is web-responsive only); paid subscriptions and monetisation; and any recommendation engine more advanced than the follow-based feed.

Chapter 2

Methodology and Design

2.1 System Architecture

InvestCircle follows a three-tier architecture: a React single-page frontend, a REST API backend with a business-services layer, and a PostgreSQL and Redis persistence layer, with background workers and the optional AI tagging microservice kept outside the request path so they can fail or lag without affecting core flows.

2.1.1 High-Level Design

Figure 2.1 shows the major components and how they interact. The three client surfaces (Discussions, News Reel, Personal) all talk to a single REST API layer, which delegates to a business services layer holding the call-resolution, feed-ranking and notification logic. Background workers sit outside the request path and communicate with external market-data and news providers.

The deployment view in Figure 2.2 maps these logical layers onto physical runtime nodes: a browser-based client, a containerised Node.js application server, a managed PostgreSQL instance, a Redis node for caching and background jobs, and the external services the application depends on.

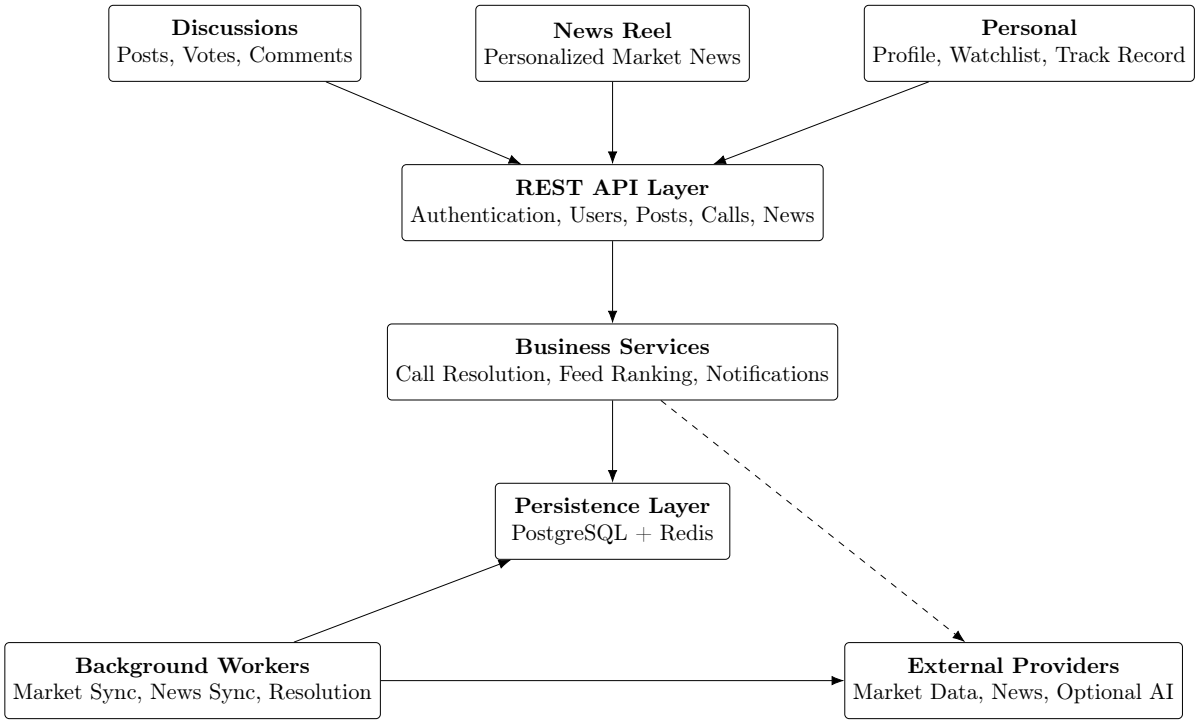


Figure 2.1: High-level layered architecture of InvestCircle.

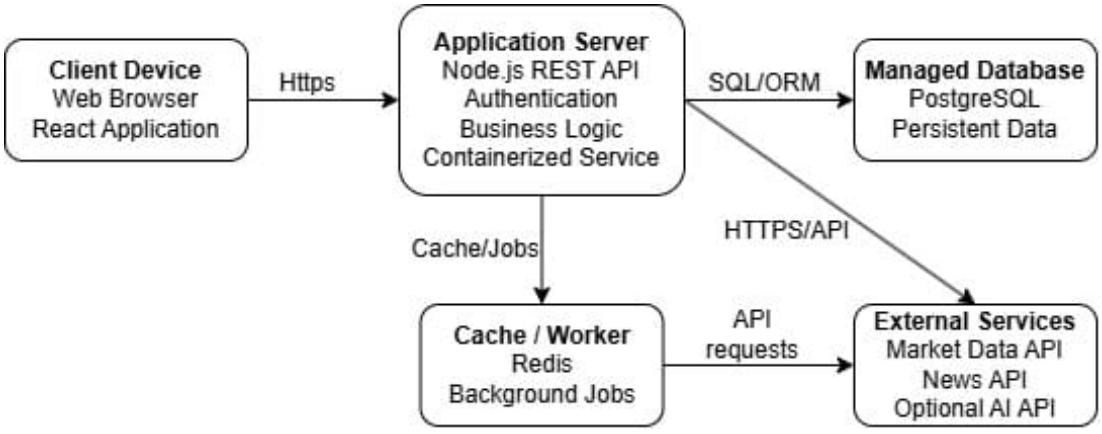


Figure 2.2: Deployment diagram — runtime nodes and their responsibilities.

Figure 2.3 identifies the actors and use cases for the system. The human actors are the **User** (who registers, publishes posts and calls, follows others, manages watchlists, comments, searches, and views notifications and accuracy scores) and the **Administrator** (who moderates content). Three automated or external actors complete the picture: the **Market Data Provider** and **News Provider**, which supply price and news data, and the **AI Service**, which asynchronously tags posts. Use cases connected with `«include»` indicate a use case that always invokes another as part of its own flow.

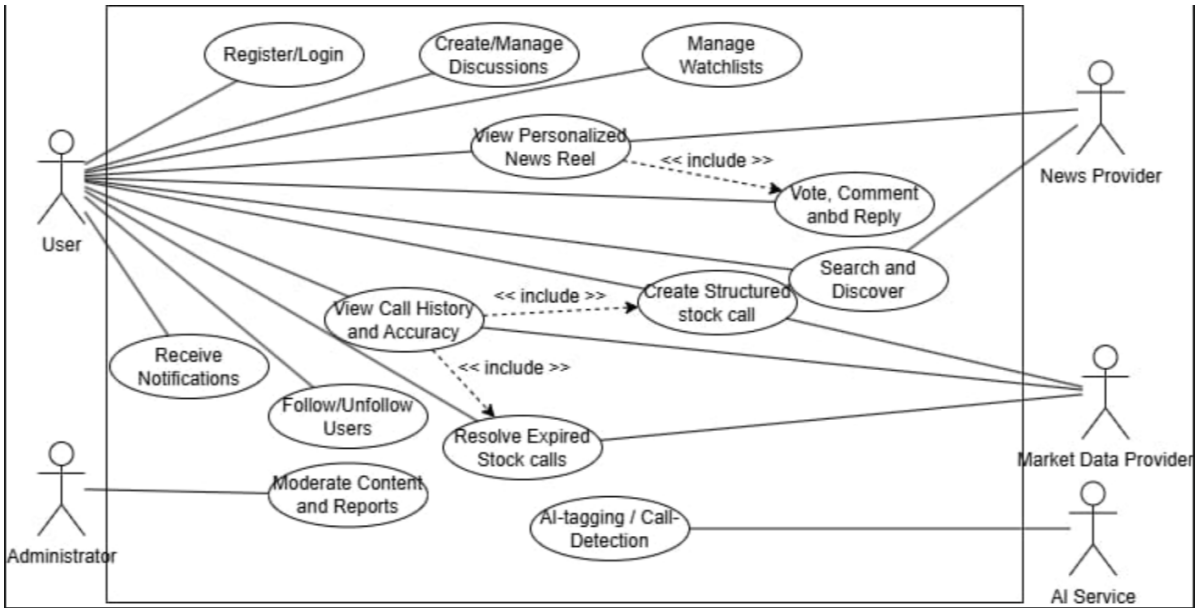


Figure 2.3: Use case diagram for InvestCircle.

The class diagram in Figure 2.4 captures the core domain model underlying the relational schema: `User`, `Post`, `Call`, `Comment`, `Follow`, `Watchlist`, `WatchlistItem` and `Notification`, along with their multiplicities.

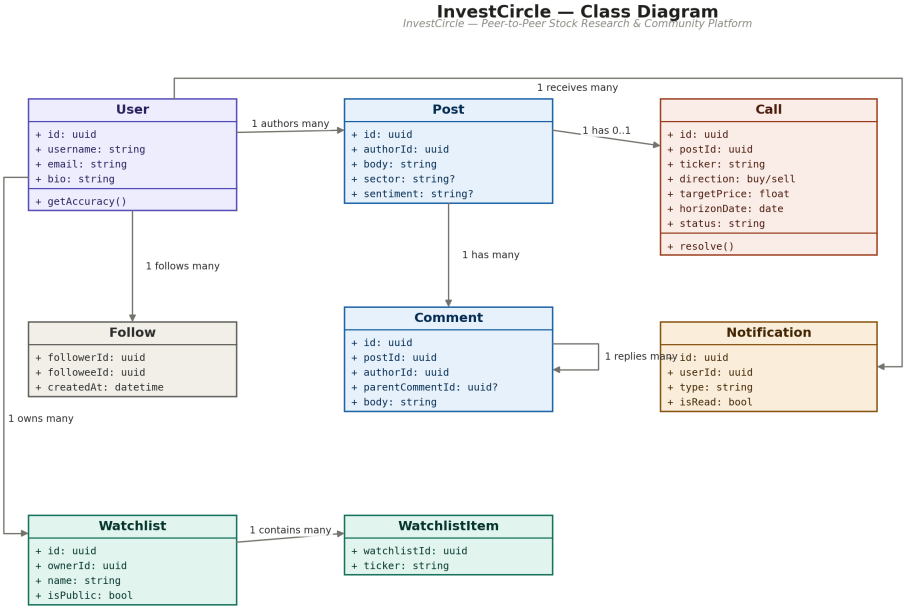


Figure 2.4: Class diagram — core domain model.

Figure 2.5 traces the create-post-with-call flow end to end. AI sector/sentiment tagging happens *asynchronously* via a Redis-backed queue: the user receives a response and sees their post in the feed immediately, while tagging happens in the background and is written back through an internal callback.

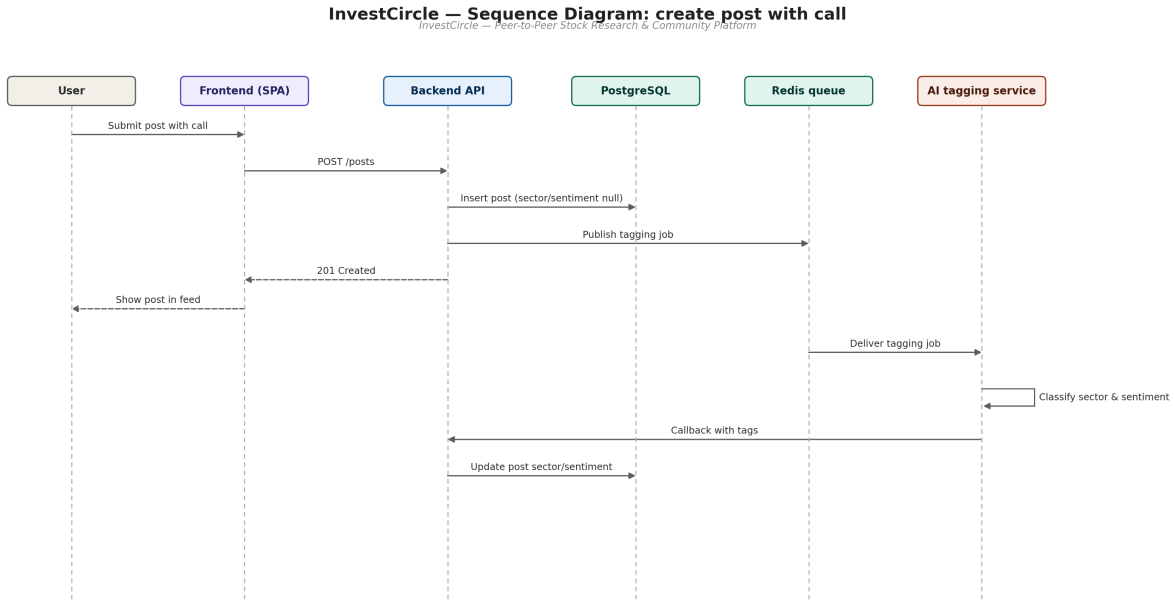


Figure 2.5: Sequence diagram — create post with a structured call.

Three activity diagrams describe the platform's core workflows. Figure 2.6 shows the creation and automatic resolution of a structured stock call — the platform's key differentiator.

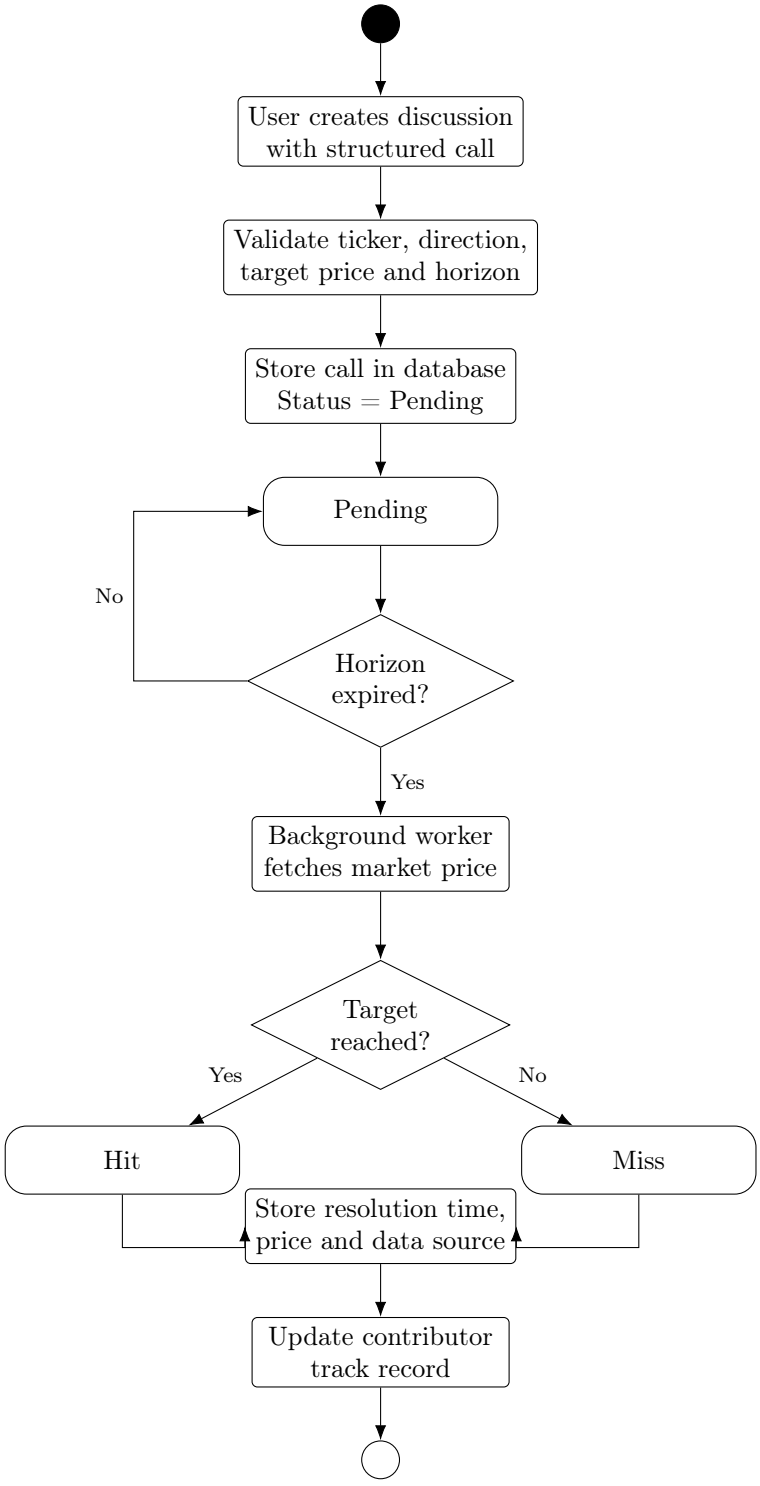


Figure 2.6: Activity diagram — creation and automatic resolution of a structured stock call.

Figure 2.7 shows the discussion workflow: composing a research post, optionally attaching a structured call, publishing, and the resulting interaction and notification generation.

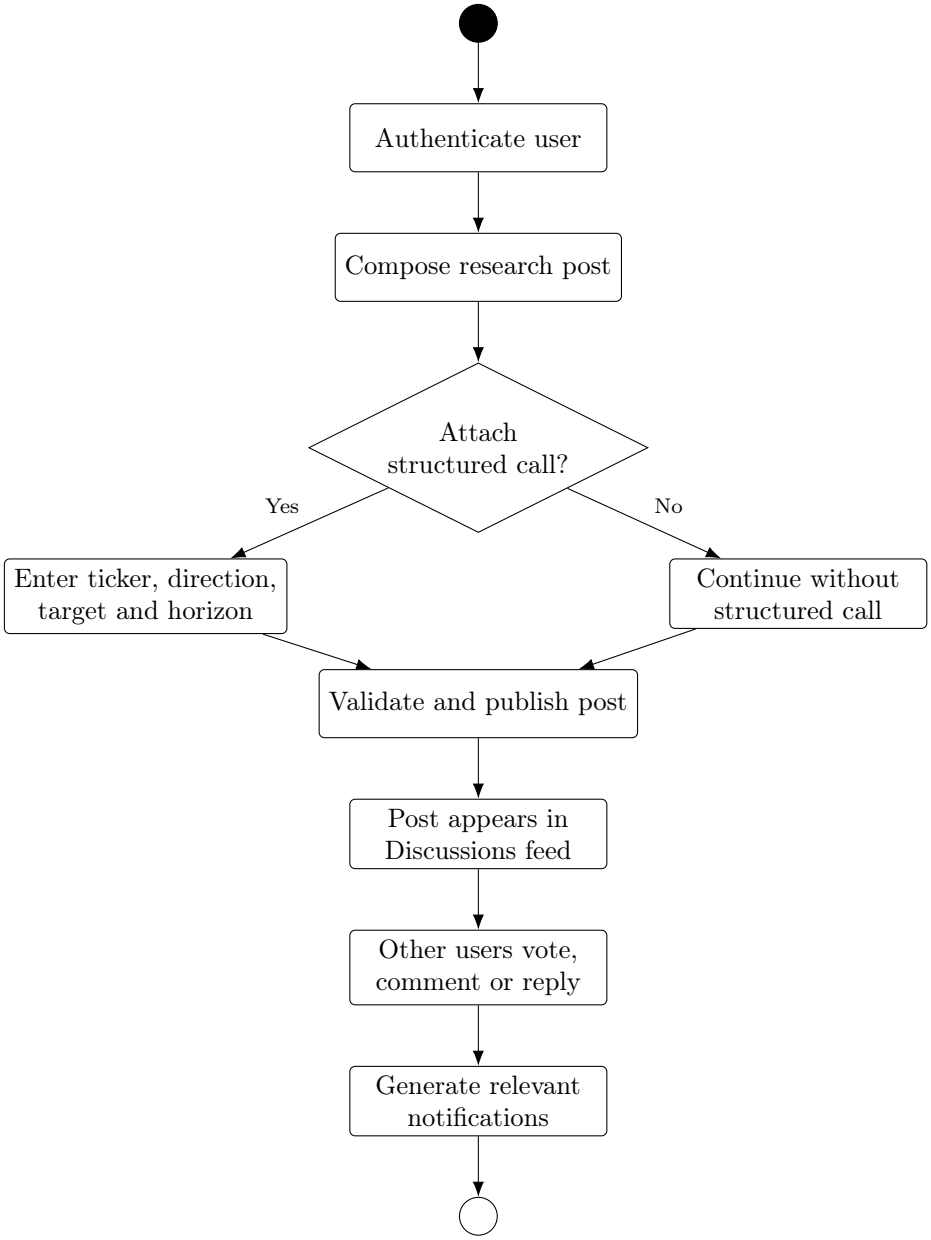


Figure 2.7: Activity diagram — publishing and interacting with a discussion.

Figure 2.8 shows how the personalised News Reel is generated and consumed, using the user's watchlist as the interest signal and deterministic relevance ranking rather than a complex recommendation model.

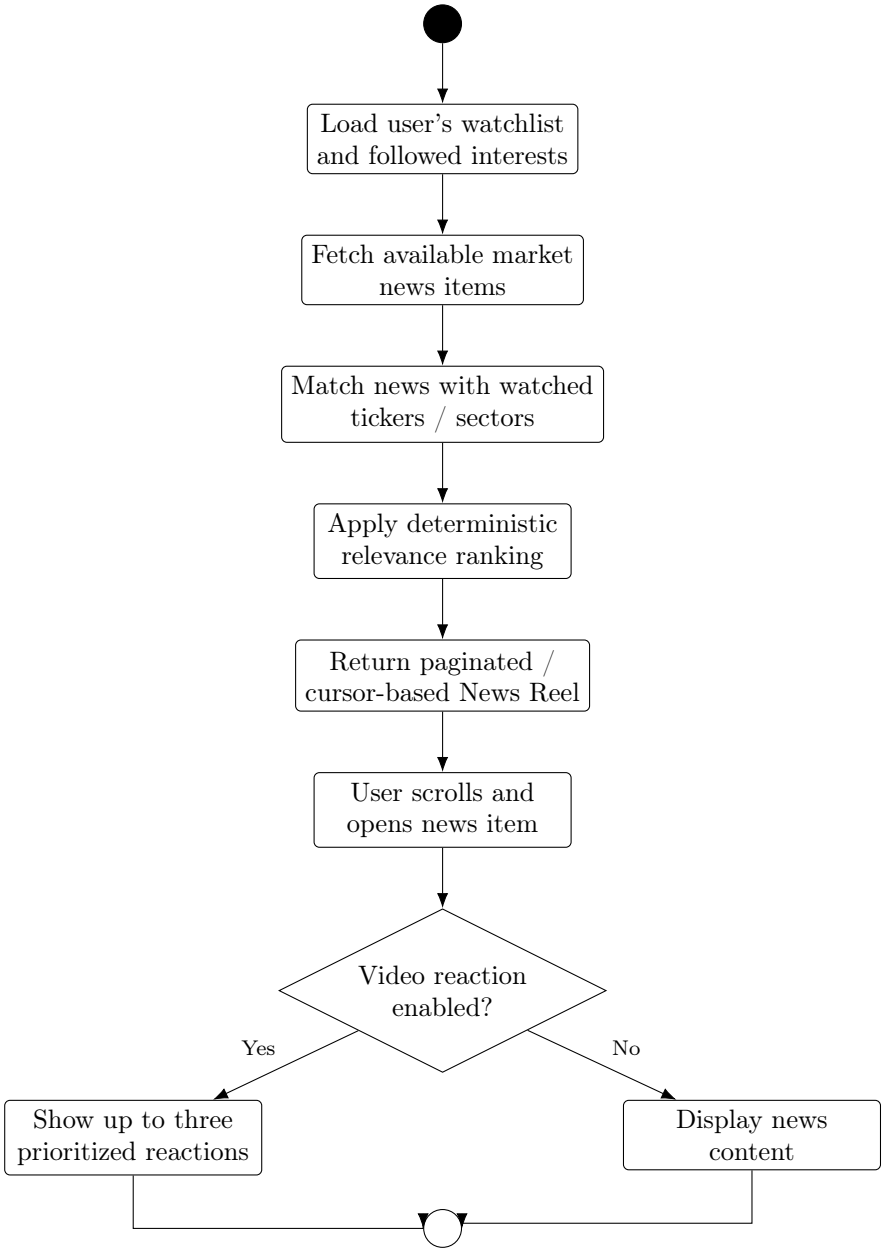


Figure 2.8: Activity diagram — generating and consuming the personalized News Reel.

The relational schema backing this design is summarised in Table 2.1. The accuracy score is computed rather than stored redundantly: it is derived from `calls.status` grouped by author via `posts.author_id`, cached per user in Redis, and recomputed whenever a call is resolved.

Table 2.1: Core database tables

Table	**Key Columns**
users	id, username, email, password_hash, bio, created_at
follows	follower_id (FK users), followee_id (FK users), created_at
posts	id, author_id (FK users), body, created_at, sector (nullable), sentiment (nullable)
calls	id, post_id (FK posts), ticker, direction (buy/sell), target_price, horizon_date, status (open/hit/miss), resolved_at
watchlists	id, owner_id (FK users), name, is_public
watchlist_items	watchlist_id (FK), ticker
comments	id, post_id (FK), author_id (FK users), parent_comment_id (nullable), body, created_at
notifications	id, user_id (FK), type, payload_json, is_read, created_at

Table 2.2 lists representative endpoints of the REST API exposed over this schema.

Table 2.2: Representative REST API endpoints

Method & Path	**Purpose**
POST /auth/register, /auth/login, /auth/refresh	Account creation and JWT session management
GET/PUT /users/:id	Profile read/update
GET /users/:id/accuracy	Accuracy score and call history
POST /follows/:userId, DELETE /follows/:userId	Follow / unfollow
POST /posts, GET /posts/:id	Create / read a post (with optional embedded call)
GET /feed?cursor=	Paginated, follow-based feed
POST /posts/:id/comments	Add a comment or threaded reply
POST /watchlists, PUT /watchlists/:id	Create / edit watchlist
GET /search?q=	Search users, tickers, posts
GET /notifications	List notifications for the current user

2.1.2 Technical Stack

- **Framework/Language:** React with TypeScript, React Query and Tailwind on the frontend (single-page, component-driven); Node.js (Express or NestJS) exposing a REST API on the backend, chosen over GraphQL for simplicity at this scale.
- **Database:** PostgreSQL, a strong relational fit for the users / posts / follows / calls model; Redis for session cache, feed cache and the background job queue (BullMQ or Celery).
- **Tools:** Docker Compose for local development, a single cloud VM or free-tier PaaS for the deployed demo, and GitHub Actions for CI running lint and unit tests on every pull request.
- **Libraries:** JWT (access plus refresh tokens) for stateless authentication with bcrypt/Argon2 password hashing; a Python FastAPI microservice wrapping a small transformer model for the optional AI tagging service; and `yfinance` for end-of-day price retrieval in the call-resolution worker.

2.2 Implementation Details

Implementation is staged across a six-month schedule (Table 2.3), with the core accountability workflow built and demonstrated first and additional features layered on only once that core is stable.

Table 2.3: Milestone plan

S.No.	Milestone
1	Requirements sign-off, system design, DB schema, API contract, repo/CI setup
2	Auth, user profiles, and watchlists — backend and frontend, deployed to staging
3	Posts, structured calls, and feed (follow-based ranking)
4	Comments, notifications, search; call-resolution background job
5	Accuracy tracking/profile stats, AI sector/sentiment tagging bolt-on, moderation basics
6	Performance/load testing, bug-fixing, deployment hardening, documentation, final demo prep

2.2.1 Module 1: Core Functionality

The core module comprises authentication, the social graph, posts with structured calls, the feed, and the call-resolution worker — that is, everything required for the accountability loop to function.

Authentication and the social graph. Accounts are created with email and password (or OAuth), with passwords hashed using bcrypt or Argon2 and never logged. Sessions use short-lived JWT access tokens (approximately 15 minutes) with rotated refresh tokens stored hashed. The follow relationship is a simple join table, indexed on both directions so that the feed query and the follower list are both cheap.

Posts and structured calls. A post is a free-text research note that may optionally carry one structured call. The call is the machine-checkable part — ticker, direction (buy or sell), target price and horizon date — validated server-side on creation. This structure is what makes later automatic resolution possible, and is the direct technical answer to the "unstructured opinions" problem identified in Section 1.

Feed. The feed is follow-based and ordered most-recent-first, retrieved through cursor-based pagination rather than offset pagination so that performance does not degrade as a user scrolls. React Query handles client-side caching and infinite scroll.

Call resolution. A scheduled background worker is the heart of the accountability mechanism. It queries calls whose horizon date has passed and whose status is still open, fetches the relevant end-of-day closing price, compares it against the target and direction, and marks the call hit or miss. Because a horizon date is not guaranteed to fall on a trading day, resolution uses a "closing price on or after this date" lookahead rather than an exact-date match. Each resolution stores the resolution timestamp and the data source used, so that any individual outcome can later be audited.

Accuracy score. The score is deliberately computed rather than stored as a denormalised column, avoiding a class of consistency bug where the stored score and the underlying call history disagree. It is derived on demand from resolved calls, cached per user in Redis, and invalidated whenever a call resolves.

User interface. The visual design concept, "Signal", uses a calm, data-dense interface — closer to a research terminal than a typical social application — in which colour is reserved almost entirely for call outcomes. Teal denotes a hit, coral a miss, and amber a call still pending; every other surface stays neutral, so that when an outcome colour appears it is immediately meaningful. The signature element is a *track-record ribbon*: a small horizontal strip of teal, amber and coral ticks displayed next to a username wherever it appears, giving an at-a-glance read of a contributor's history without opening their profile. Headings use Space Grotesk, body text IBM Plex Sans, and all tickers, prices and timestamps IBM Plex Mono with tabular figures enabled so that numbers stay aligned and scannable.

Table 2.4: Colour palette

Name	Hex	Usage
Ink	#14181F	Primary text, nav bar, headers — the neutral anchor
Paper	#F4F6F5	Application background
Signal Teal	#1FA98C	Primary action colour and "Hit" call outcome
Alert Coral	#E1583F	"Miss" call outcome, destructive actions
Amber	#E8A23D	"Pending" call status, follow/notify accents
Slate	#5B6572	Secondary text, borders, icons

Figure 2.9 shows how the core screens connect from the application shell, and Figure 2.10 traces the end-to-end pipeline from signup through the accountability loop.

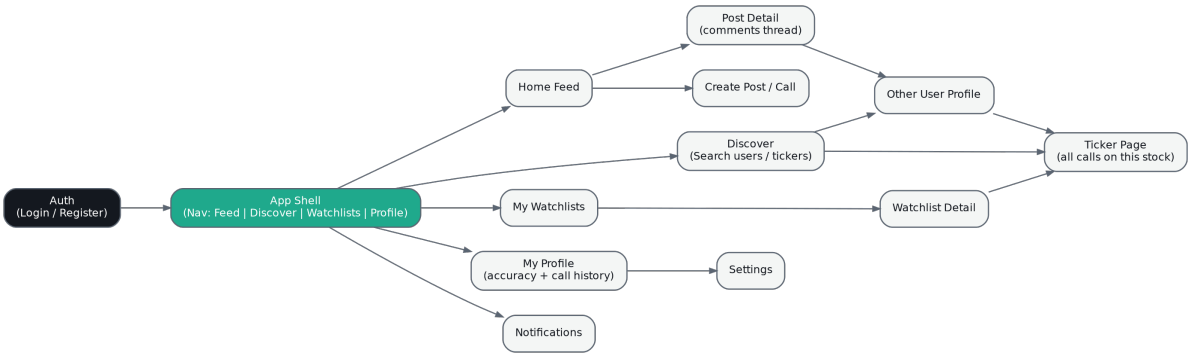


Figure 2.9: Screen map — navigation between core screens.

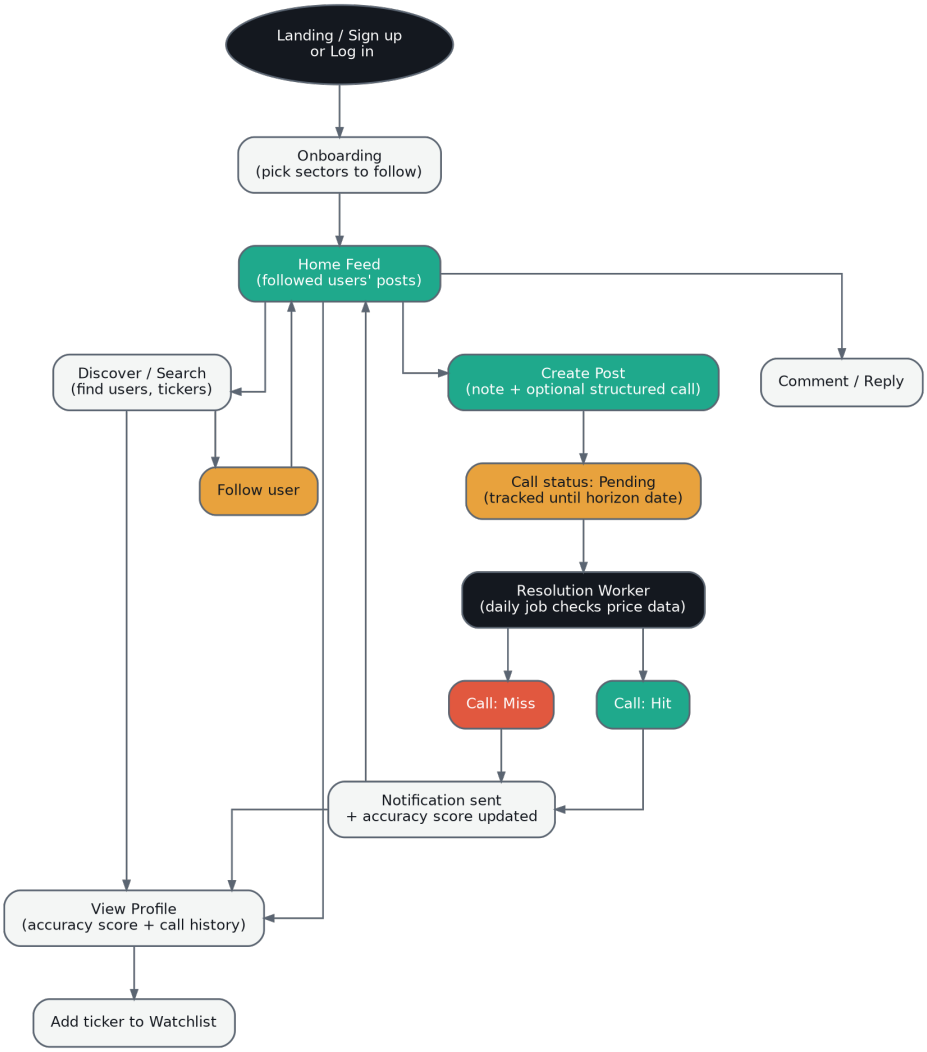


Figure 2.10: Application flow — signup through call resolution and accuracy update.

2.2.2 Module 2: Secondary Features

The secondary module contains everything that enriches the platform but that the core loop does not depend on: notifications, search, the personalised News Reel, moderation, and the AI tagging service.

Notifications. Notifications are written synchronously to the notifications table on the triggering event (a new follower, a comment on your post, or a call resolving) and fetched by client polling in the first release; a websocket push is a stretch goal.

Search and discovery. Search spans users, tickers and posts using PostgreSQL full-text search, with indexes on the frequently queried attributes (user IDs, tickers and timestamps).

News Reel. The News Reel is a vertically scrolling feed of market news, personalised by treating each user's watchlist as an interest signal. Ranking is deliberately deterministic — items matching a user's watched tickers and sectors are boosted — rather than using a complex recommendation model, which keeps the behaviour explainable and testable. Critically, the external finance API is never called from the frontend: a backend cron job pulls from the provider and caches results in Redis, which bounds the request rate against a rate-limited external service and keeps the client fast. Figure 2.11 shows these integration points.

AI sector and sentiment tagging. This is the project's single AI feature and is deliberately isolated. On post creation the API publishes a message to a Redis-backed queue; a separate FastAPI microservice consumes it, classifies the post body by sector and sentiment, and writes the result back through an internal callback endpoint. If the service is unavailable, the post is still created immediately with sector and sentiment left null — the feature is additive and never blocking. This design directly satisfies Objective 4.

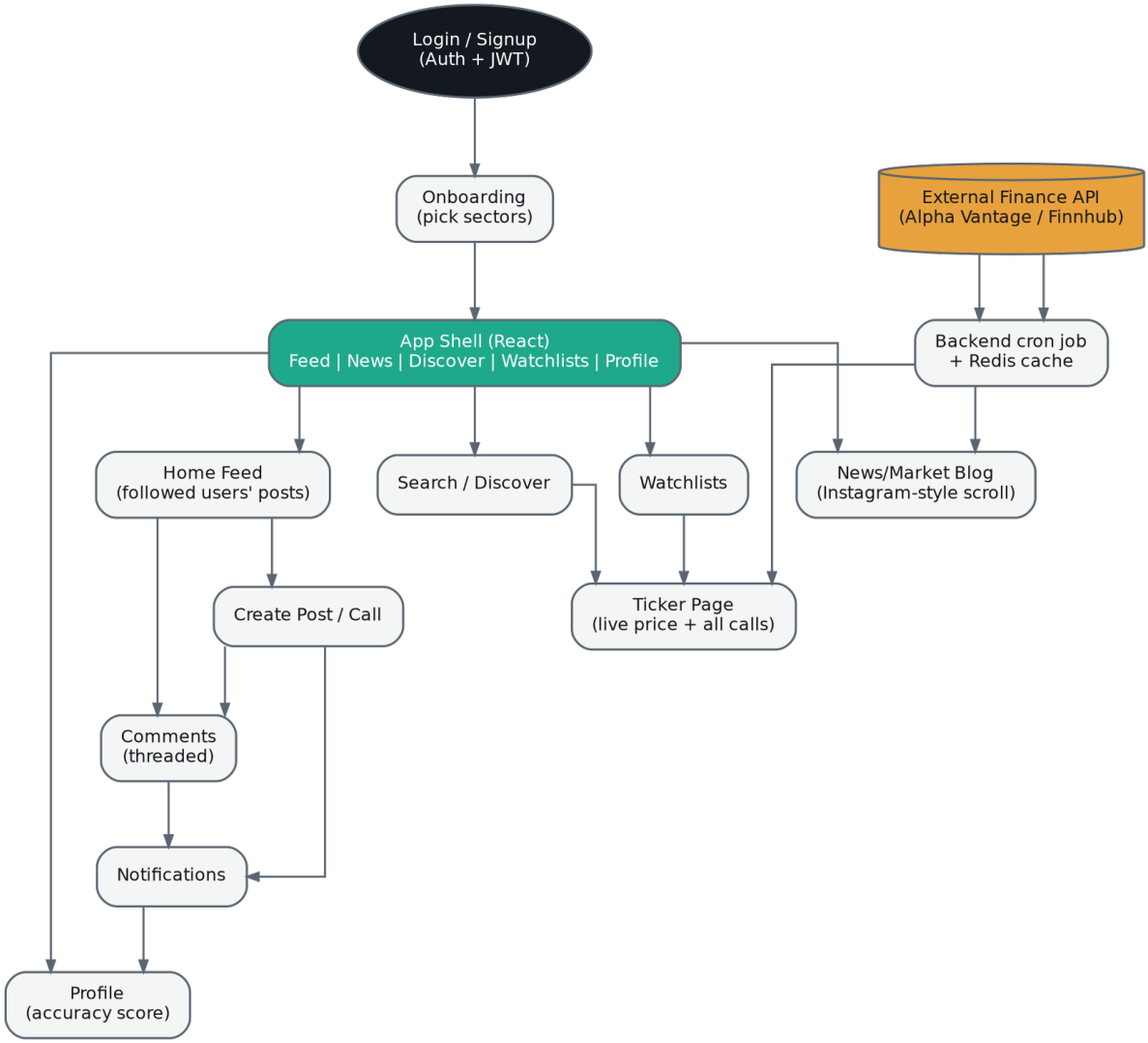


Figure 2.11: Feature flowchart — screens and external API integration points.

Security measures. Across both modules, input validation and parameterised queries (via the ORM) are applied on all endpoints to prevent SQL injection; rate limiting is applied to authentication and post-creation endpoints; and CORS is restricted to the deployed frontend origin.

Test data. Because the schema holds entirely user-generated content that does not exist yet, and because using real users' data would be inappropriate for a course project, a synthetic seed dataset was generated to support development and demonstration: 60 users, a follow graph of roughly 470 edges, 150 research posts, 92 structured calls, 220 threaded comments, 129 watchlists holding 841 items, and 478 derived notification events, all produced with a fixed random seed for reproducibility. A separate balanced set of 240 labelled samples (80 positive, 80 neutral, 80 negative, covering all reference tickers) was built specifically to test the AI tagging service independently of the seed posts. Both are accompanied by a full PostgreSQL seed script matching the schema in Table 2.1.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The testing strategy covers four levels:

- **Unit Testing:** Business logic is tested in isolation, with particular focus on the two pieces of logic where a silent error would be most damaging — the accuracy calculation and the call-resolution comparison. The resolution function is written as pure logic, taking a call record and a price and returning an outcome, with no I/O of its own, specifically so that it can be exhaustively unit tested without a database or a network call. The target is at least 70% coverage of the backend business-logic layer.
- **Integration Testing:** Core API endpoints (authentication, posts, follow, feed) are tested against a real database instance to verify that the schema, the ORM layer and the endpoint contracts agree.
- **Performance Testing:** The feed and profile endpoints are load-tested against the seeded dataset described in Section 2.2, which is sized to represent a realistic project-scale workload.
- **User Testing:** A manual QA pass over the core user flows (post, follow, comment, view accuracy) is carried out before each monthly milestone review.

A key validation exercise was performed on the call-resolution logic using real market data rather than synthetic prices. A backtest set of 200 structured calls was generated directly from actual NSE trading history: for each call, a real historical trading day was chosen first, the target price was computed from the *real* closing price on that day, and the hit/miss outcome was computed against the *real* closing price on the horizon date. This provides an independently verifiable check that the resolution logic produces correct outcomes against real market behaviour, not merely against numbers the project generated itself.

3.2 Performance Metrics

Table 3.1 states the project's target metrics and the status of each at the current stage of work.

The backtest produced a hit rate of 20% (40 of 200 calls). This figure is unforced — it is simply what results from selecting random real entry points and directions with targets 5–20%

Table 3.1: Evaluation metrics: targets and current status

Metric	Target	Status
Call-resolution reliability	$\geq 95\%$ of due calls resolved automatically	Logic validated on 200 real-price calls; 100% resolved without manual intervention
Feed load time	< 1.5 s (up to 200 follows)	Pending implementation; seeded dataset prepared for load testing
Backend test coverage	$\geq 70\%$	Pending implementation
Core availability without AI service	100% of core flows	Satisfied by design (queue-based, nullable tags)
Real-data ticker coverage	Reference ticker set	32 of 35 tickers covered

away from the entry price, evaluated against real price history. It is reported here not as a performance target but as evidence that the resolution mechanism discriminates correctly between outcomes rather than trivially returning one label.

3.3 Analysis

Did the work meet its objectives? Objective 4 (AI strictly secondary) is satisfied by design and verifiable from the architecture: the tagging service is reached only through a queue, and the `sector` and `sentiment` columns are nullable, so a post is created successfully whether or not the service ever responds. Objective 2 is partially demonstrated — the resolution logic is implemented and validated against real market data, though the scheduled worker is not yet running against a live feed. Objectives 1 and 3 are in progress, with the design, schema, API contract and supporting datasets complete and implementation scheduled across Months 2–6.

Where the work exceeded expectations. The original plan was to validate call resolution against synthetic prices and integrate real market data later. Locating a usable public source of NSE daily OHLCV data made it possible to validate the logic against genuine market history considerably earlier than scheduled, which converts what would have been an assumption into a measured result.

Data provenance. Being precise about what is real and what is generated matters for interpreting these results:

- **Real:** the 32-ticker daily OHLCV price history used for the backtest, covering January 2000 to April 2021. Every price and date is unmodified real market data.
- **Synthetic but grounded in real data:** the 200 backtest calls. No public dataset of "structured stock calls with tracked outcomes" exists — that is precisely the artefact InvestCircle is designed to produce — so the calls themselves were generated, but every price against which they are judged is real.
- **Fully synthetic:** the seed dataset (users, follow graph, posts, comments, watchlists, notifications and the AI tagging samples), used to populate a realistic current-dated

instance of the application for demonstrating screens end to end.

Challenges encountered and how they were resolved. Three are worth recording. First, three of the project's 35 reference tickers (`APOLLOHOSP`, `HDFCLIFE`, `SBILIFE`) are absent from the historical dataset, as they were not index constituents as of its cutoff; the backtest was scoped to the 32 covered tickers rather than substituting approximate data. Second, a horizon date frequently falls on a weekend or market holiday, so a naive exact-date price lookup fails; this was resolved with the on-or-after lookahead described in Section 2.2. Third, the historical dataset and the synthetic seed data occupy disjoint date ranges (2000–2021 against 2024–2026), so merging them would have left nearly every synthetic call unmatched; rather than force a merge, the real-price backtest was kept as a separate self-contained validation set, which is also the more honest framing when reporting results.

Preprocessing applied. The raw historical dataset required decompression from its distributed archive; filtering from 50 stocks down to the 32 intersecting the project's reference list; type conversion of dates and prices, which arrive as text; the trading-day lookahead described above; and column selection, since only the closing price is required for resolution. The raw data has known gaps in its `Trades` and `Deliverable Volume` columns for some early-2000s rows; this did not affect the results above, as those columns are unused, but it is recorded here for transparency.

Chapter 4

Conclusion

4.1 Summary of Work

The objective of this project is to build a peer-to-peer stock research platform in which credibility is measured rather than claimed — where every structured call a contributor publishes is automatically checked against real market data and rolled into a permanent public accuracy record.

At the current stage, the requirement specification and system design are complete. The functional and non-functional requirements are finalised; the relational schema and REST API surface are defined; and the architecture, deployment, use case, class, sequence and activity views presented in Chapter 2 give a consistent and implementable picture of the system. The UI/UX design system and screen flows translate those requirements into a concrete interface organised around the platform's central idea. Supporting datasets — a synthetic seed set for demonstration, a labelled sample set for the AI tagging service, and real NSE price data for validation — are in place ahead of implementation, so that development, testing and the first demonstration do not block on data availability.

Against the four objectives stated in Section 1: Objective 4 is met and verifiable from the architecture; Objective 2 is substantially demonstrated, with the resolution logic validated against real market history; and Objectives 1 and 3 are in progress on the milestone schedule in Table 2.3.

4.2 Key Findings

- **Finding 1:** Making a prediction *structured* rather than free-text is the single design decision that makes accountability mechanisable. Once a call carries a ticker, direction, target and horizon, resolution becomes a deterministic comparison that a background job can perform without human judgement — which is exactly what every platform surveyed in Chapter 1 lacks.
- **Finding 2:** Computing the accuracy score on demand, rather than storing it as a column, removes an entire class of consistency bug. A stored score can drift out of agreement with the call history that is supposed to justify it; a derived and cached score cannot.
- **Finding 3 (unexpected):** Real market data proved substantially easier to obtain than

anticipated, allowing validation of the resolution logic months ahead of schedule. Conversely, an unexpected obstacle emerged in the opposite direction — the mismatch between the historical data range and the synthetic seed data range meant the two datasets could not be combined, and keeping them separate turned out to be both simpler and more honest than engineering a merge.

- **Finding 4:** Isolating the AI feature behind a queue with nullable result columns meant that "the system works when the AI is down" required no special handling, error recovery or fallback path — it is a consequence of the data model rather than a feature that had to be built.

4.3 Future Work and Recommendations

1. **Complete the core implementation:** build the backend (authentication, posts, calls, feed, follow graph) and the call-resolution and notification workers per the Month 2–4 milestones, followed by the frontend screens per the design system.
2. **Integrate a live market-data provider:** move the call-resolution worker from historical CSV data to a live end-of-day feed for tickers and the News Reel, retaining the provider abstraction so that sources can be swapped without touching business logic.
3. **Wire in the AI tagging microservice** using the labelled sample set already prepared, keeping it strictly off the critical request path.
4. **Scalability improvements:** the pull-based feed is appropriate at project scale, but a fan-out-on-write model would be worth exploring should the followed-set cardinality grow substantially; similarly, the notification polling in the first release would be replaced by websocket push.
5. **Real-world deployment considerations:** a public deployment would require attention to regulatory framing — the platform hosts opinions, not advice, and would need clear disclaimers to avoid being construed as investment advisory — alongside moderation tooling and abuse handling at a scale beyond the basic report-and-hide feature currently scoped.
6. **Extended evaluation:** contributor rankings, sector-level accuracy breakdowns, and detection of AI-inferred implied calls from free-text posts (labelled distinctly so they are never silently mixed into a user's confirmed record) are natural extensions once the core record is established.

4.4 Final Remarks

The most useful lesson from this stage of the project concerns the relationship between data modelling and product intent. InvestCircle's entire premise — that credibility should be earned rather than asserted — turns out to rest almost entirely on a schema decision: representing a prediction as four typed fields instead of a paragraph of text. Every downstream capability, from automatic resolution to the accuracy score to the track-record ribbon in the interface, follows from that one choice. Conversely, the features that proved most fragile in planning were the ones where the data model was vaguest.

A second lesson concerns honesty about evidence. It would have been straightforward to present the synthetic seed data as though it demonstrated the system working, and considerably harder

to locate real market data and report a hit rate of 20% that flatters nothing. The latter is the more defensible engineering position, and drawing a clear line between what was measured and what was generated has made the project's claims easier to justify rather than harder.

Bibliography

[1] National Stock Exchange of India (NSE). *Nifty 50 index constituents and sector classification.* https://www.nseindia.com/.

[2] Rao, R. *NIFTY-50 Stock Market Data (2000–2021).* Kaggle Datasets. https://www.kaggle.com/datasets/rohanrao/nifty50-stock-market-data.

[3] Periwal, N. *NSE India Stock Data (1990–2021).* Kaggle Datasets. https://www.kaggle.com/datasets/stoicstatic/india-stock-data-nse-1990-2020.

[4] *yfinance — Python library for Yahoo Finance market data.* https://pypi.org/project/yfinance/.

[5] OWASP Foundation. *Password storage and authentication best practices (bcrypt/Argon2, JWT).* https://owasp.org/.

[6] PostgreSQL Global Development Group. *PostgreSQL Documentation: Full Text Search and Indexing.* https://www.postgresql.org/docs/.